

Praktikum 13: VueJS Autentikasi dan Navigation Guards (SPA Security)

Tujuan

1. Mahasiswa mampu memahami konsep keamanan dan pembatasan hak akses rute pada sisi klien (*Client-Side Security*).
2. Mahasiswa mampu memahami konsep *Navigation Guards* (*beforeEach*) pada Vue Router.
3. Mahasiswa mampu membuat API Endpoint autentikasi pada backend CodeIgniter 4.
4. Mahasiswa mampu mengimplementasikan modul Login dan proteksi halaman admin pada aplikasi *Single Page Application* (SPA) Frontend API.

Instruksi Praktikum

1. Persiapkan text editor misalnya Visual Studio Code (VSCode).
2. Pastikan web server Apache dan MySQL pada XAMPP dalam kondisi aktif.
3. Buka folder proyek backend CodeIgniter 4 (*lab7_php_ci*) dan folder frontend VueJS (*lab8_vuejs*) yang telah dibuat pada praktikum sebelumnya.
4. Ikuti langkah-langkah praktikum secara berurutan.

Teori Singkat

Apa itu Navigation Guards?

Dalam aplikasi web tradisional berbasis server-side (seperti MVC standar CodeIgniter), proteksi halaman dilakukan menggunakan *Filters* atau *Middleware* sebelum halaman HTML dirender oleh server. Namun, pada arsitektur *Single Page Application* (SPA), seluruh struktur halaman web sudah dimuat di awal oleh browser klien.

Untuk mengamankan rute-rute internal tertentu (seperti halaman /artikel) agar tidak bisa dibuka oleh pengguna yang belum melakukan autentikasi, Vue Router menyediakan fitur **Navigation Guards** melalui fungsi `router.beforeEach()`. Fungsi ini bertindak sebagai interceptor (pencegat) perpindahan rute yang akan memeriksa status login pengguna (misalnya memeriksa keberadaan token login di `localStorage`) sebelum mengizinkan rute tersebut ditampilkan ke layar browser.

Langkah-langkah Praktikum

TAHAP 1: PEMBUATAN API ENDPOINT LOGIN (SISI BACKEND CI4)

Sebelum masuk ke konfigurasi antarmuka klien, kita harus menyediakan endpoint REST API di CodeIgniter 4 untuk memvalidasi kredensial pengguna yang dikirim dari aplikasi frontend.

Langkah 1.1: Membuat Auth Controller Buat berkas controller baru pada direktori proyek backend Anda di `app/Controllers/Api/Auth.php`, kemudian ketikkan kode berikut:

PHP

```
<?php

namespace App\Controllers\Api;

use CodeIgniter\RESTful\ResourceController;
use App\Models\UserModel;

class Auth extends ResourceController
{
    protected $format = 'json';

    public function login()
    {
        // 1. Menerima data input dari request body JSON
        $username = $this->request->getVar('username');
        $password = $this->request->getVar('password');

        $model = new UserModel();

        // 2. Cari user berdasarkan username atau email di database
        $user = $model->where('username', $username)
            ->orWhere('useremail', $username)
            ->first();

        if ($user) {
            // 3. Verifikasi password (sesuaikan dengan format enkripsi
            database Praktikum 4)
            if ($password === $user['userpassword'] ||
            password_verify($password, $user['userpassword'])) {

                // Jika sukses, kirim status data dan token respon
                return $this->respond([
                    'status' => 200,
                    'error' => null,
                    'messages' => 'Login Berhasil',
                    'data' => [
                        'id' => $user['id'],
                        'username' => $user['username'],
                        'token' => base64_encode("TOKEN-SECRET-" .
                        $user['username'])
                    ]
                ], 200);
            }
        }
    }
}
```

```
    }

    // 4. Jika gagal, kirim status error 401 (Unauthorized)
    return $this->failUnauthorized('Username atau Password yang Anda
masukkan salah.');
```

Langkah 1.2: Mendaftarkan Route API Login Buka berkas konfigurasi routing backend Anda di app/Config/Routes.php, lalu daftarkan rute khusus penanganan proses login dengan metode POST:

PHP

```
$routes->post('api/login', 'Api\Auth::login');
```

TAHAP 2: PENGEMBANGAN INTEGRASI FRONTEND (SISI VUEJS SPA)

Sesuaikan struktur direktori pada folder proyek frontend lab8_vuejs Anda agar memiliki berkas komponen Login.js yang baru seperti struktur berikut:

```
lab8_vuejs/
├── index.html
├── assets/
│   ├── css/
│   │   └── style.css
│   ├── js/
│   │   ├── app.js
│   │   └── components/
│   │       ├── Home.js
│   │       ├── Artikel.js
│   │       └── Login.js
```

Langkah 2.1: Membuat Komponen Login (assets/js/components/Login.js) Buat file baru bernama Login.js di dalam folder components/. Komponen ini berfungsi menampilkan form login, merekam input, dan mengirimkannya ke API Backend menggunakan Axios.

JavaScript

```
const Login = {
  template: `
    <div class="login-container">
      <div class="login-box">
        <h2>Form Login Admin</h2>
        <form @submit.prevent="handleLogin">
          <div class="form-group">
            <label>Username / Email</label>
```

```

        <input type="text" v-model="username"
placeholder="Masukkan username" required>
    </div>
    <div class="form-group">
        <label>Password</label>
        <input type="password" v-model="password"
placeholder="Masukkan password" required>
    </div>
    <button type="submit" class="btn-login">Masuk
Aplikasi</button>
    </form>
    <p v-if="errorMessage" class="error-msg">{{ errorMessage
}}</p>
    </div>
</div>
},
data() {
    return {
        username: '',
        password: '',
        errorMessage: ''
    }
},
methods: {
    handleLogin() {
        // Mengirim data kredensial ke API Endpoint backend CI4
        axios.post(apiUrl + '/api/login', {
            username: this.username,
            password: this.password
        })
        .then(response => {
            if (response.data.status === 200) {
                // Simpan status otentikasi riil dari server ke local
storage browser
                localStorage.setItem('isLoggedIn', 'true');
                localStorage.setItem('userToken',
response.data.data.token);

                // Alihkan halaman ke manajemen artikel secara programatis
this.$router.push('/artikel');
                window.location.reload();
            }
        })
        .catch(error => {
            // Tangkap pesan error dari response backend jika gagal login
            if (error.response && error.response.data.messages) {
                this.errorMessage = error.response.data.messages;
            } else {
                this.errorMessage = 'Terjadi kesalahan jaringan atau

```

```
server. ';\n\n    }\n  });\n}\n};
```

Langkah 2.2: Mengonfigurasi Proteksi Rute dan Guards pada assets/js/app.js

Buka berkas app.js. Daftarkan komponen Login, tambahkan properti meta: { requiresAuth: true } pada rute artikel, serta buat fungsi kontrol beforeEach.

JavaScript

```
const { createApp } = Vue;\nconst { createRouter, createWebHashHistory } = VueRouter;\n\n// Tentukan lokasi API REST End Point sesuai konfigurasi local proyek Anda\nconst apiUrl = 'http://localhost/labci4/public';\n\n// 1. Definisikan mapping rute URL ke Komponen beserta properti Meta-Auth\nconst routes = [\n  { path: '/', component: Home },\n  { path: '/login', component: Login },\n  {\n    path: '/artikel',\n    component: Artikel,\n    meta: { requiresAuth: true } // Hanya boleh diakses jika user sudah\nlogin\n  }\n];\n\nconst router = createRouter({\n  history: createWebHashHistory(),\n  routes\n});\n\n// 2. Implementasi Navigation Guards (Pencegat Akses Rute)\nrouter.beforeEach((to, from, next) => {\n  const isAuthenticated = localStorage.getItem('isLoggedIn') === 'true';\n\n  // Jika rute tujuan membutuhkan autentikasi dan user belum login\n  if (to.matched.some(record => record.meta.requiresAuth) &&\n!isAuthenticated) {\n    alert('Akses Ditolak! Anda harus login terlebih dahulu.');
```

```
// 3. Inisialisasi Root Instance dengan State Navigasi Global
const app = createApp({
  data() {
    return {
      isLoggedIn: false
    }
  },
  mounted() {
    // Cek status login saat aplikasi pertama kali dimuat oleh browser
    this.isLoggedIn = localStorage.getItem('isLoggedIn') === 'true';
  },
  methods: {
    logout() {
      if (confirm('Apakah Anda yakin ingin keluar aplikasi?')) {
        localStorage.removeItem('isLoggedIn');
        localStorage.removeItem('userToken');
        this.isLoggedIn = false;
        this.$router.push('/');
      }
    }
  }
});

app.use(router);
app.mount('#app');
```

Langkah 2.3: Menyesuaikan Tata Letak Dinamis pada index.html Buka file index.html utama. Muat skrip komponen Login.js dan tambahkan direktif v-if / v-else pada menu navigasi bagian atas untuk merubah link login/logout secara dinamis.

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Secured SPA Frontend VueJS</title>
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  <script src="https://unpkg.com/vue-router@4/dist/vue-router.global.js"></script>
  <script src="https://unpkg.com/axios/dist/axios.min.js"></script>
  <link rel="stylesheet" href="assets/css/style.css">
</head>
<body>
  <div id="app">
    <header>
      <h1>Aplikasi Panel Single Page (SPA) - Secured</h1>
```

```
        <nav class="nav-menu">
            <router-link to="/">Beranda</router-link> |
            <router-link to="/artikel">Kelola Artikel</router-link> |

            <router-link v-if="!isLoggedIn" to="/login">Login</router-
link>
            <a v-else href="#" @click.prevent="logout">Logout</a>
        </nav>
    </header>

    <main style="margin-top: 20px;">
        <router-view></router-view>
    </main>
</div>

<script src="assets/js/components/Home.js"></script>
<script src="assets/js/components/Artikel.js"></script>
<script src="assets/js/components/Login.js"></script>
<script src="assets/js/app.js"></script>
</body>
</html>
```

Langkah 2.4: Menambahkan Desain Antarmuka Form pada assets/css/style.css

Tambahkan pengaturan kode CSS berikut pada bagian paling bawah file stylesheet Anda agar tampilan kotak formulir login berada rapi di tengah halaman web:

CSS

```
.login-container {
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 40px 0;
}
.login-box {
    width: 350px;
    padding: 25px;
    border: 1px solid #ccc;
    border-radius: 8px;
    background-color: #ffffff;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}
.login-box h2 {
    margin-top: 0;
    margin-bottom: 20px;
    text-align: center;
    color: #333;
}
.form-group {
```

```
        margin-bottom: 15px;
    }
    .form-group label {
        display: block;
        margin-bottom: 5px;
        font-weight: bold;
    }
    .form-group input {
        width: 100%;
        padding: 8px;
        border: 1px solid #ccc;
        border-radius: 4px;
        box-sizing: border-box;
    }
    .btn-login {
        width: 100%;
        padding: 10px;
        background-color: #3152d6;
        color: white;
        border: none;
        border-radius: 4px;
        font-weight: bold;
        cursor: pointer;
    }
    .btn-login:hover {
        background-color: #203ca3;
    }
    .error-msg {
        color: red;
        font-size: 14px;
        text-align: center;
        margin-top: 15px;
    }
}
```

Pertanyaan dan Tugas

1. Selesaikan seluruh pengerjaan kode pemrograman sistem autentikasi API di atas.
2. Jalankan pengujian skenario kontrol keamanan berikut pada browser:
 - **Skenario A (Kondisi Terkunci):** Bersihkan penyimpanan local storage browser (atau dalam keadaan belum login). Tekan menu navigasi "Kelola Artikel". Amati jalannya aplikasi, apakah sistem berhasil menolak akses secara langsung, memunculkan alert, dan melempar halaman tampilan ke form login.
 - **Skenario B (Kondisi Login Terautentikasi):** Buka form login, masukkan data akun pengguna yang valid sesuai isi database user Anda. Amati apakah sistem berhasil memvalidasi kredensial ke database backend melalui Axios, membawa Anda masuk ke halaman tabel artikel, serta merubah tombol menu login atas menjadi link "Logout".

3. Terapkan pengaman rute serupa (`meta: { requiresAuth: true }`) untuk komponen halaman `About.js` (profil mahasiswa) yang telah dibuat pada Praktikum 12, sehingga menu tersebut ikut terproteksi dari pengguna luar.

Laporan Praktikum

1. Melanjutkan penulisan riwayat praktikum pada repository GitHub yang sama dengan nama `Lab11Web_VueJS`.
2. Ambil gambar screenshot jalannya program di web browser pada saat proses penolakan rute, pengisian formulir, respon sukses login, dan setelah menekan fungsi logout.
3. Tuliskan penjelasan analisis ringkas mengenai alur kerja fungsional dari `router.beforeEach` dan `Axios HTTP Post` pada berkas dokumentasi `README.md`.
4. Lakukan proses commit dan push pembaruan seluruh berkas kode pengerjaan Praktikum 13 ini ke dalam repository GitHub Anda.
5. Kirimkan tautan URL repositori tersebut ke dalam sistem `ecampus` Universitas Pelita Bangsa sebagai bukti pengumpulan tugas.